

Підготовка до захисту

Програмна система електронного навчання програмуванню з автоматизованою перевіркою практичних завдань

Булах М. Р. · Науковий керівник: Лукашов О. В. · Київ, 2026

Мета та завдання роботи

Мета

Підвищення ефективності електронного навчання програмуванню та зменшення навантаження на викладачів за рахунок автоматизованої перевірки практичних завдань і генерації персоналізованих підказок на основі штучного інтелекту.

Завдання

1. Проаналізувати предметну область, наявні платформи онлайн-навчання програмуванню та сформулювати вимоги до системи.
2. Спроекувати архітектуру програмної системи та модель даних.
3. Розробити підсистему реєстрації та автентифікації (email/пароль і Google OAuth 2.0) з розмежуванням ролей.
4. Реалізувати підсистему керування контентом — курси, модулі та уроки трьох типів — і редактор курсу для викладача.
5. Інтегрувати вбудований редактор коду Monaco з ізольованою компіляцією та виконанням C++ через Godbolt.
6. Розробити механізм автоматичної перевірки рішень за тест-кейсами з порівнянням виводу.
7. Інтегрувати ШІ-ментор на базі Google Gemini для генерації персоналізованих українськомовних підказок.
8. Реалізувати інструменти адміністрування системи.
9. Провести тестування та апробацію системи серед користувачів і проаналізувати результати.

Частина 1. Доповідь (нотатки до слайдів, ≈5 хв)

1. Титульний. Доброго дня, шановна комісіє! Мене звати Марина Булах. Тема моєї кваліфікаційної роботи — «Програмна система електронного навчання програмуванню з автоматизованою перевіркою практичних завдань». Науковий керівник — Олександр Володимирович Лукашов. (~15 с)

2. Актуальність. Актуальність зумовлена кількома чинниками. Перехід до дистанційної освіти потребує платформ, які навчають програмуванню без постійної присутності викладача. Ручна перевірка коду забирає в педагогів багато часу. Студенту потрібен миттєвий зворотний зв'язок — одразу бачити, де помилка. А штучний інтелект відкриває нові можливості для персоналізованого навчання. Саме на перетині цих потреб побудована моя система. (~30 с)

3. Спрямованість дослідження. Мета — підвищення ефективності електронного навчання програмуванню та зменшення навантаження на викладачів за рахунок автоматизованої перевірки практичних завдань і генерації персоналізованих ШІ-підказок. Об'єкт — процес електронного навчання програмуванню; предмет — методи, архітектура та інструменти інтеграції модулів у єдину веб-платформу. (~30 с)

4. Вимоги до системи. Серед функціональних — реєстрація через email і Google OAuth, структуровані курси з уроками трьох типів, редактор Monaco з компіляцією через Godbolt, інтеграція з Gemini для підказок українською, інструменти викладача й адміністратора. Серед нефункціональних — безпека на основі bcrypt, JWT і HTTPS, ізольоване виконання коду, асинхронний інтерфейс і сучасний UI. (~30 с)

5. Діаграма використання. Три ролі: студент проходить курси, виконує завдання й отримує ШІ-підказки; викладач створює курси та тестові сценарії; адміністратор керує користувачами й ролями. Окремо винесено вхід через Google OAuth як розширення стандартної автентифікації. (~20 с)

6. Клієнт-серверна взаємодія. Архітектура клієнт-серверна з чітким поділом відповідальності. Клієнт на Next.js і React містить редактор Monaco. Бекенд на NestJS відповідає за авторизацію, курси та приймання рішень. Дані — у PostgreSQL через Prisma. Компіляція й виконання C++ — ізольовано через GCC, а персоналізовані підказки дає Google Gemini. Такий поділ робить систему масштабованою й безпечною. (~35 с)

7. Діаграма діяльності. Описує повний цикл проходження модуля. Студент вивчає теорію і проходить квіз; за помилок — повертається на повторне навчання. Далі практика: код компілюється через Godbolt, і при помилці генерується персоналізована ШІ-підказка. Потім запускаються тести. Усі три результати сходяться в єдиній кінцевій точці — відповідно до нотації UML. (~30 с)

8. ШІ-Ментор на базі Gemini. (ключовий слайд — не поспішай) Це інноваційна складова роботи. Ментор не дає готової відповіді й не вказує на помилку напряду, а наводить навідними запитаннями. Приклад: пропущено крапку з комою. Компілятор видає сухе «error: expected semicolon», що лякає початківця. А ментор запитує: «чим у C++ завершується кожна окрема інструкція?» — і студент сам знаходить помилку. Тобто система не просто перевіряє, а навчає. (~35 с)

9. Інтерфейс студента. Зліва — умова задачі та підказки, у центрі — редактор коду, знизу — консоль із тестами. Студент пише код, натискає «Run», бачить результат по кожному тесту й лише після проходження всіх може здати завдання. (~25 с)

10. Редактор курсу (викладач). Викладач створює модулі й уроки, пише теорію в зручному редакторі з живим попереднім переглядом, додає спеціальні блоки на кшталт карток. Усе одразу стає доступним студентам. (~20 с)

11. Конструктор тест-кейсів. Викладач задає умову задачі, шаблон коду, рівень складності та набір тест-кейсів із вхідними даними й очікуваним результатом. Саме за цими тестами система автоматично перевіряє рішення. (~20 с)

12. Публікація. За темою дослідження опубліковано тези доповіді на 14-й Східноєвропейській конференції MSTIoE 2026 у Київському національному університеті імені Тараса Шевченка. (~15 с)

13. Апробація системи. Систему апробовано серед 25 школярів 8–10 класів, які повністю пройшли курс основ C++. Учні відзначили миттєву перевірку коду, зрозумілі підказки ШІ українською, зручний редактор у браузері та покрокову структуру з прогресом. Серед побажань — більше прикладів до складних тем, ширша база завдань, гейміфікація та мобільна версія. (~30 с)

14. Висновки та перспективи. Розроблено закінчену освітню веб-платформу для вивчення C++, готову до впровадження. Реалізовано безпечне виконання коду в ізольованому середовищі, автоматичну перевірку з порівнянням виводу, а ШІ-ментор Gemini знизив відсоток відмов від складних завдань. У перспективі — підтримка Python, Java, Rust, розширення бази завдань і мобільний застосунок. (~30 с)

15. Дякую за увагу. Дякую за увагу! Готова відповісти на ваші запитання. (~10 с)

Частина 2. Відповіді на можливі питання комісії

1. Чому саме Google Gemini?

Я обирала ШІ-модель за чотирма критеріями: вартість, швидкість, якість української мови та простота інтеграції.

- **Вартість.** Gemini має безкоштовний tier, достатній для освітнього проєкту без бюджету; OpenAI GPT — платний і потребує прив'язки картки.
- **Швидкість.** Я використовую `gemini-flash-latest` — модель із низькою затримкою. Це критично, бо підказка має з'явитися за секунди.
- **Українська мова.** Одна з вимог — підказки природною українською; Gemini якісно її підтримує.
- **Інтеграція.** Офіційний SDK дав швидку й надійну інтеграцію.

Архітектурно ШІ винесений в окремий сервіс, тому модель можна замінити без переписування системи.

Якщо спитають «чому не локальна модель»: локальні моделі потребують власного GPU-сервера, що неможливо в межах безкоштовного хостингу.

2. Яким чином відбувається інтеграція з ШІ?

Інтеграція **повністю серверна** — через окремий сервіс `AiService` на бекенді NestJS, який використовує офіційний SDK `@google/generative-ai`. API-ключ зберігається на сервері й **ніколи не потрапляє в браузер**. Потік такий:

- Студент відправляє рішення.
- Бекенд виконує код у пісочниці Godbolt по всіх тест-кейсах.
- Якщо тест не пройдено (помилка компіляції/виконання або невідповідність виводу) — система звертається до Gemini.
- У запит передається контекст: код студента, тип помилки, очікуваний і фактичний вивід, зміст уроку.
- Gemini повертає підказку, яку бекенд віддає у картку «ШІ-ментор».

Ключовий момент — **промпт-інжиніринг**: модель отримує інструкцію бути ментором, давати підказку на 1–2 речення українською й не давати готового коду. Адміністратор може ввімкнути/вимкнути ШІ та замінити ключ; за недоступності сервісу є запасне повідомлення.

3. Яким чином тестували коректність підказок ШІ?

Варто розрізнити коректність **оцінювання** і коректність **підказки**.

- **Оцінювання** повністю детерміноване: правильність рішення визначають тест-кейси через точне порівняння виводу, а не ШІ. Тому навіть неідеальна підказка не впливає на оцінку — це принциповий запобіжник.
- **Підказки** я тестувала емпірично: склала набір типових помилок початківців (пропущена `;`, відсутній `#include`, неоголошена змінна, `=` замість `==`, неправильний формат виводу) і перевіряла, що підказка стосується правильної помилки, не містить готового коду та зрозуміла українською.
- Фінальне підтвердження — апробація на 25 учнях, які окремо відзначили зрозумілість і користь підказок.

ШІ свідомо винесений у допоміжну, а не вирішальну роль.

Якщо спитають «а що як ШІ дасть неправильну підказку»: підказка не блокує й не оцінює — студент орієнтується на результати тестів; у найгіршому разі підказка просто менш корисна, але шкоди не завдає.

4. Як забезпечується ізоляція та безпека виконання чужого коду?

Захист побудований на двох рівнях.

- **Ізоляція коду.** Код студента компілюється й виконується **не на моєму сервері**, а у зовнішній пісочниці Godbolt (Compiler Explorer, GCC). Тому потенційно небезпечний чи помилковий код фізично не має доступу до мого бекенду, а Godbolt сам обмежує час і пам'ять виконання.
- **Безпека доступу.** Паролі хешуються через bcrypt, сесії — на основі JWT, передавання даних — лише через HTTPS.

Якщо спитають «а що як код зациклиться чи з'їсть пам'ять»: Godbolt має власні ліміти часу й пам'яті, і оскільки це не мій сервер — нашкодити системі такий код не може.

5. Чому саме C++, а не Python?

- **Цільова аудиторія.** Система орієнтована на школярів і абітурієнтів, а C++ входить до шкільної програми з інформатики та олімпіадного програмування в Україні.
- **Педагогічна цінність.** Суворі типізація і строгий компілятор змушують студента розуміти основи — типи, синтаксис, структуру програми, — а не лише «отримати результат».
- **Синергія з ШІ.** Саме «суворі» помилки компілятора C++ — те місце, де ментор приносить найбільше користі, перетворюючи незрозумілу помилку на навідну підказку.

Рівень перевірки (порівняння виводу) не залежить від мови, тому додавання Python, Java чи Rust закладено як перспектива розвитку.

6. Як працює автоматична перевірка рішень?

Перевірка — це чорноскринькове тестування за входом-виходом. Для кожного тест-кейсу:

- Код студента з вхідними даними надсилається на Godbolt, де компілюється й виконується.
- Якщо код повернув ненульовий код завершення — це помилка компіляції/виконання: тест провалено, генерується ШІ-підказка.
- Інакше фактичний вивід програми порівнюється з очікуваним. Перед порівнянням обрізаються зайві пробіли й переноси (нормалізація), далі — точне порівняння рядків.
- Усі тест-кейси мають пройти. Перший провалений зупиняє перевірку зі статусом «не пройдено»; якщо всі успішні — прогрес зберігається.

7. Чим система відрізняється від готових платформ (LeetCode, Moodle)?

- **LeetCode / HackerRank** — алгоритмічні тренажери англійською для досвідчених розробників і співбесід. Там немає теорії, навчання та україномовного сократичного ментора.
- **Moodle** — загальна LMS: вона не вміє «з коробки» компілювати й автоматично перевіряти C++, не має вбудованого редактора коду й ШІ-підказок.

Моя система — це **унікальне поєднання в одній україномовній платформі**: структурована теорія, вбудований редактор Monaco, автоматична перевірка та сократичний ШІ-ментор, орієнтовані на початківців і абітурієнтів. Це не «ще один онлайн-судья», а навчальне середовище.

8. Як масштабується система?

- **Винесення обчислень.** Найважче — компіляція й виконання коду — делеговано зовнішньому Godbolt, тому мій сервер лишається «легким» і не стає вузьким місцем за CPU.
- **Stateless-архітектура.** API не зберігає стан сесії (автентифікація через JWT), тому його можна запускати в кількох екземплярах горизонтально.
- **Асинхронність.** Інтерфейс не блокується під час перевірки рішення.
- **Черга.** Redis закладено в інфраструктуру як чергу для майбутнього асинхронного оброблення рішень; за поточного навантаження компіляція виконується синхронно через Godbolt.

Для себе (не озвучувати зайвого): на слайдах Redis показано як активний компонент, але в коді він поки не задіяний. Якщо спитають конкретно про Redis — відповідай саме формулюванням вище («зкладено як чергу для майбутнього асинхронного оброблення»), не стверджуй, що він уже обробляє рішення.